70016 Natural Language Processing Coursework Report

Yanli Wang[1,2,3]

[1]CID: 02215760, Department of Computing, yanli.wang22@imperial.ac.uk
[2]Github Repository: https://github.com/Yi1lan/NLP-Lifecycle-Research
[3]Leaderboard: Yi1lan

March 4, 2026

Contents

1 Task Definition and Literature Review

Patronizing and Condescending Language (PCL) describes discourse that often implicitly positions the speaker as socially or morally superior while portraying vulnerable communities through pity, moralizing charity, or stereotyped assumptions, frequently framed as well-intentioned support [1]. *Don't Patronize Me!* [2] argues that PCL is technically distinct from overt hate or abuse: it is typically subtler, more subjective, and context dependent, motivating dedicated resources and evaluation tasks. Concretely, the paper introduces a dataset with span-level annotations and benchmarks two problems: (i) paragraph-level binary detection of PCL and (ii) paragraph-level prediction of PCL categories.

1.1 Primary Contribution

The primary contributions of *Don't Patronize Me!* are fourfold:

- **Resource for PCL detection towards vulnerable communities:** They introduce the *Don't Patronize Me!* dataset containing 10,637 news paragraphs (2010–2018) sampled from the News on Web (NoW) corpus [3] using ten vulnerability-related keywords, with sampling balanced across keyword–country combinations.

- **Span-level supervision and task formulation:** Beyond paragraph labels, the dataset provides span annotations for PCL and associates spans with fine-grained categories, enabling both binary detection and category-based modelling.
- **Two-level taxonomy of PCL phenomena:** The paper proposes seven PCL categories organized into three higher-level families (i.e., *The Saviour*, *The Expert*, *The Poet*) with *Other*, intended to operationalize distinct mechanisms of condescension and support more diagnostic evaluation.
- **Baseline benchmarking for two tasks:** The paper provides reproducible baseline results across classical models and fine-tuned Transformers for both binary PCL detection and category prediction, establishing a reference point for future systems.

1.2 Technical Strengths

Don't Patronize Me! demonstrates significant technical merit through its rigorous data collection and experimental benchmarking, which can be summarized as:

- **Annotation protocol designed for subjectivity:** The authors report Kappa IAA $\kappa = 0.41$ across the three paragraph-level labels $(0/1/2)$ between the two main annotators; when paragraphs marked borderline by either annotator are removed, agreement increases to $\kappa = 0.61$. They also report the underlying disagreement profile ($9,182$ agreement vs. $1,457$ disagreements, including 590 total disagreements), making annotation ambiguity explicit rather than hidden.
- **Expert annotation with adjudication:** The dataset is annotated by three expert annotators (backgrounds in communication, media, and data science); two annotate the full dataset, and the third acts as a referee to resolve total disagreements, which improves label reliability for a subtle pragmatic phenomenon.
- **Broad and transparent baseline benchmarking:** The paper evaluates classical and neural baselines under 10-fold cross-validation and reports positive-class precision/recall/F1 for Task 1 (e.g., RoBERTa F1 $= 70.63$ vs. Random F1 $= 33.31$). Hyperparameters and reproducibility choices are described (e.g., random seed fixed to 1 for fine-tuned LMs), which makes the benchmark useful as a reference point.
- **Diagnostic qualitative analysis:** The paper includes qualitative examples of model errors (e.g., incorrect predictions made by RoBERTa) and explicitly argues that some cases require world knowledge/commonsense (e.g., assessing whether a proposed solution is shallow or whether a presupposition is warranted), which helps diagnose what current models miss.

1.3 Key Weaknesses

Despite its contributions, the work suffers from limitations regarding data reliability and experimental depth:

- **Moderate agreement and label noise remain a bottleneck:** While the annotation design is thoughtful, the reported agreement indicates substantial ambiguity in many instances. Concretely, the paper reports $\kappa = 0.41$ across the three labels, and $1,457$ disagreements between the two main annotators (including 590 total disagreements), which shows that ambiguity is frequent enough to materially affect training and evaluation unless it is handled explicitly.
- **Severe imbalance and category sparsity are insufficiently addressed:** Interpreted as binary, the dataset contains only 995 positive PCL paragraphs; moreover, the paper itself notes that *The poorer, the merrier* is the least represented category (64 samples) and that this can explain the poor results for that category, highlighting how sparsity limits reliable fine-grained modeling.
- **Span annotations are underutilized in modelling:** Despite span-level labels being a key dataset contribution, in the Task 2 baseline setup, the authors explicitly state that they treat Task 2 as *paragraph-level multi-label classification* and therefore *span boundaries are not used as part of the training data*, leaving the value of span supervision under-tested.
- **Potential sampling bias from keyword-based retrieval:** Since paragraphs are retrieved using ten vulnerability-related keywords and balanced across keyword-country combinations, models may learn topic/keyword-adjacent cues rather than PCL as a discourse phenomenon; the paper does not provide controlled evidence (e.g., counterfactual edits or cross-domain tests) to rule out such shortcuts.

- **Limited uncertainty reporting in the experimental results:** While the experiments use 10-fold cross-validation and the fine-tuned LMs are run with a fixed random seed 1, the reported results are presented as single P/R/F1 values (no standard deviations/CI/significance tests), making it hard to judge whether small differences between strong Transformer baselines are robust.

Verdict (Weak Accept): The paper makes a valuable contribution by releasing a clearly-defined dataset and benchmarking suite for a subtle pragmatic phenomenon, which is likely to catalyze follow-up work. However, the evidence for generalizable modeling gains is limited by moderate annotation ambiguity and by baselines that do not exploit the dataset's span-level supervision, leaving important methodological questions open.

2 Data Acquisition, Exploration, and Preprocessing

In this section, we perform exploratory data analysis (EDA) on the training dataset `dontpatronizeme_pcl.tsv`[1] to identify dataset properties that directly impact patronizing and condescending language (PCL) detection. To keep the EDA operational, each analysis is paired with a concrete modelling implication (e.g., loss design, sampling, input configuration, or robustness checks). We examine three complementary aspects: (i) **class imbalance**, which affects gradient signal, decision thresholds, and the stability of positive-class F1 (the official metric); (ii) **sequence length distribution**, which determines the maximum input length and truncation strategy and can influence whether discourse-level cues are preserved; and (iii) **lexical signal analysis (unigram frequency)**, which quantifies whether surface tokens provide strong class-specific cues and motivates lexical baselines and shortcut-risk evaluations.

2.1 Class Distribution Analysis

Figure 1 presents the distribution of the binary labels in the official training split:

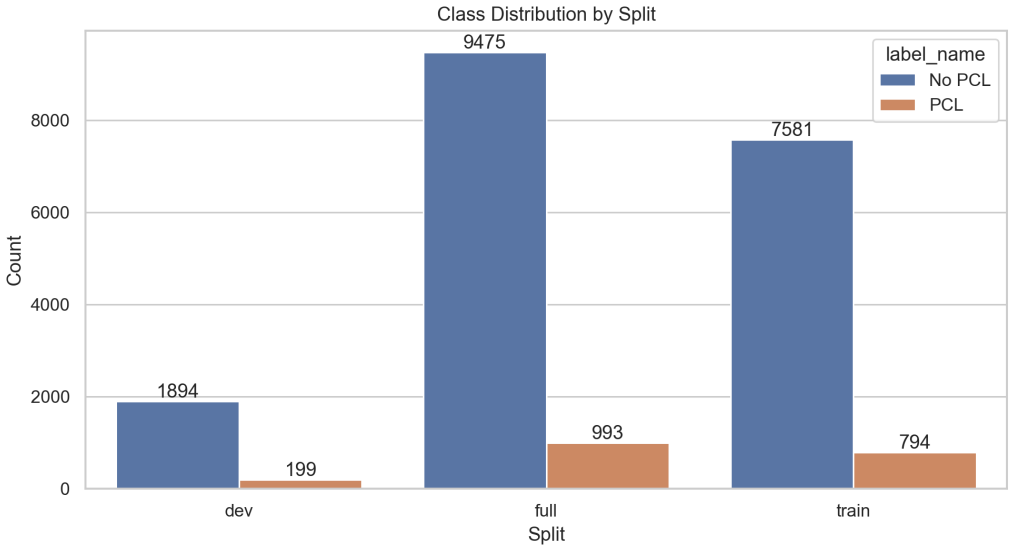

Figure 1: Class distribution of PCL vs. Non-PCL examples in `dontpatronizeme_pcl.tsv`.

Class	Count	Percentage
No PCL (0)	9,475	90.5%
PCL (1)	993	9.5%
Total	**10,468**	**100.0%**

Table 1: Class distribution for paragraph-level PCL detection in `dontpatronizeme_pcl.tsv`.

[1]Unless stated otherwise, all statistics below are computed on the provided official training split ($n = 10,468$), which is a subset of the full dataset described in the original paper (10,637 paragraphs). Dataset available at: https://github.com/CRLala/NLPLabs-2024/blob/main/Dont_Patronize_Me_Trainingset/dontpatronizeme_pcl.tsv

Analysis. Table 1 shows a pronounced skew: only 993 of 10,468 paragraphs are labelled as PCL, yielding a minority prevalence of 9.5%.[2] Numerically, this means a trivial majority classifier would achieve 90.5% accuracy by predicting No PCL for every paragraph ($9,475/10,468$), so accuracy would be misleading; this motivates focusing on positive-class F1 and on procedures that increase recall without collapsing precision.

Impact statement. This imbalance directly informs both evaluation and training design:

- **Evaluation criterion and reporting:** Model selection should be driven primarily by the positive-class F1 (PCL), reported alongside precision, recall, and the confusion matrix, as overall accuracy is not informative under severe class imbalance.
- **Operating-point selection:** The decision threshold will be treated as a tunable parameter rather than fixed at 0.5; it should be selected on the validation set to optimize positive-class F1 (or an explicitly stated precision–recall trade-off appropriate for the task).
- **Imbalance-aware training objective:** Given the 9.5% skew in Table 1, introducing cost-sensitive objectives (e.g., class-weighted cross-entropy or focal loss) could improve positive-class recall and hence positive-class F1 by amplifying gradient signal from minority examples. A controlled comparison of unweighted CE vs. weighted CE vs. focal would be a clean way to test whether any observed gains come from (i) better recall, (ii) better precision, or (iii) a changed decision boundary.
- **Stratified validation protocol:** Under severe imbalance, estimated F1 can vary noticeably across random splits. If extensive hyperparameter tuning is required, constructing an internal validation split via stratified sampling could reduce variance in the estimated positive-class F1 by preserving prevalence, enabling more reliable model comparisons.

2.2 Sequence Length Distribution

We examine the distribution of paragraph lengths to inform the choice of input truncation and padding for downstream classifiers. Length is measured as the number of whitespace-delimited tokens, which provides a coarse but model-agnostic proxy for input size; in practice, subword tokenization (e.g., BPE/WordPiece) can increase sequence lengths, so the analysis below should be interpreted as a lower bound on encoder token budgets. Figure 2 visualizes the corresponding density distributions:

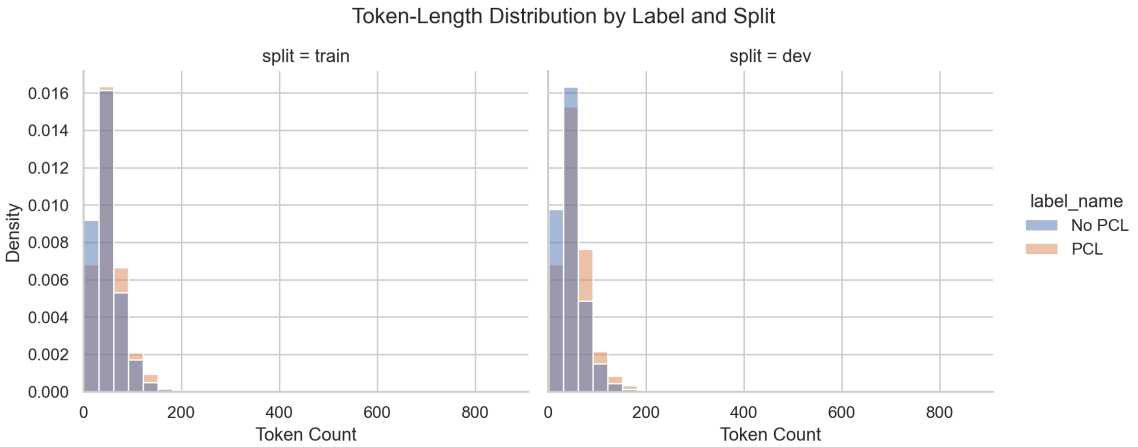

Figure 2: Length distribution (token density) for PCL vs. non-PCL paragraphs.

Analysis. Table 2 summarizes corpus-level statistics. PCL paragraphs are systematically longer than non-PCL paragraphs (mean 53.62 vs. 47.88 tokens; median 47 vs. 42). Concretely, 10% of PCL paragraphs exceed 90 whitespace tokens and 5% exceed about 113 tokens, so a tight truncation budget would disproportionately cut context for a measurable fraction of positive examples. While both classes contain rare outliers (max 909 tokens for non-PCL; 512 for PCL), the percentile statistics indicate that extremely long inputs are atypical.

[2] This differs slightly from the full dataset totals reported by Perez Almendros et al. [2] (995 positives) since we analyze the provided training split rather than the entire corpus.

Class	Count	Mean	Median	P90	P95	Max
No PCL (0)	9,475	47.88	42	83	100	909
PCL (1)	993	53.62	47	90	112.8	512

Table 2: Whitespace-token length statistics by class (from `length_summary.csv`).

Impact statement. These findings motivate an input-length policy that preserves contextual cues for PCL while remaining computationally efficient:

- **Truncation budget:** Given that the positive class exhibits a longer tail, overly restrictive cut-offs risk disproportionately truncating PCL instances. For instance, since P90 $= 90$ for the PCL class, any limit below 90 tokens (including 64) would truncate at least 10% of positive examples under whitespace tokenization; we therefore avoid such low cut-offs.
- **Model configuration:** Since subword tokenization typically increases token counts relative to whitespace tokens, selecting `max_seq_length` via a small sweep (e.g., 192 vs. 256) could yield a better compute-coverage trade-off. In particular, using a larger limit (e.g., closer to 256) may reduce truncation-induced false negatives on longer positive examples, while a smaller limit may reduce padding overhead; the sweep would determine which regime improved positive-class F1 in practice.

2.3 Lexical Signal Analysis (Unigram Frequency)

To test whether the two labels are separable using *surface-level* lexical cues (as opposed to deeper pragmatic or discourse structure), we compute unigram statistics on the training split and quantify token–class association. Concretely, after basic text normalization (lowercasing and removing punctuation/URLs), we count token occurrences per class. Let $c_{w,y}$ denote the count of token w in class $y \in \{0, 1\}$ and let $N_y = \sum_w c_{w,y}$ be the total token count for class y. We define a *class advantage score* as the (absolute) log-odds contrast of the smoothed relative rates:

$$s(w) = \left| \log \frac{\tilde{p}(w \mid y{=}1)}{\tilde{p}(w \mid y{=}0)} \right|, \qquad \tilde{p}(w \mid y) = \frac{c_{w,y} + \alpha}{N_y + \alpha V}, \tag{1}$$

where V is the vocabulary size and $\alpha > 0$ is a small additive smoothing constant (to stabilize scores for low-frequency tokens, whereas in our experiment $\alpha = 1$). Intuitively, $s(w)$ is large when a token's *relative rate* differs substantially across classes; the sign of the (unsymmetrized) log-rate ratio indicates which class the token favors. Figure 3 shows the most distinctive unigrams for each class:

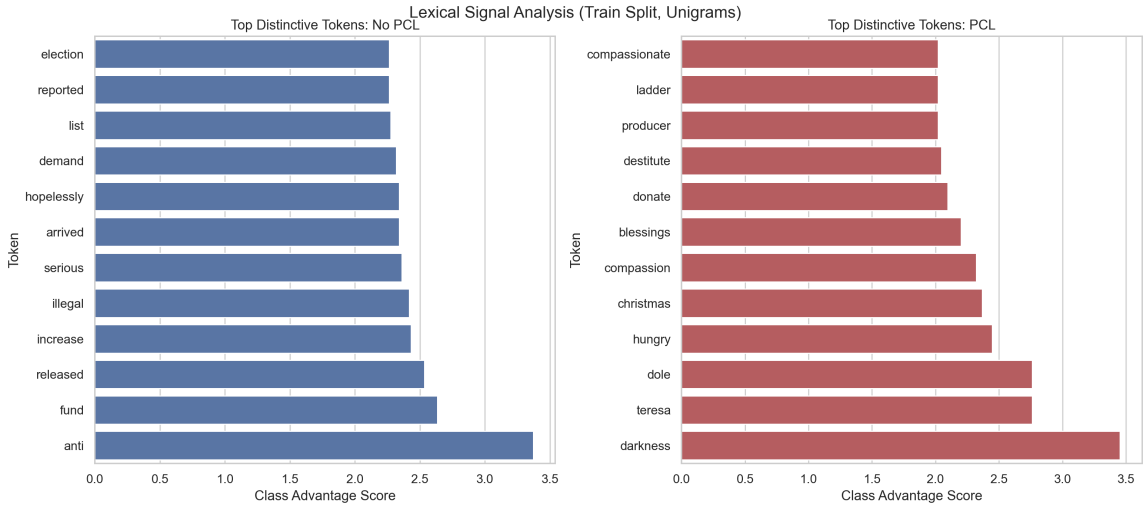

Figure 3: Lexical signal analysis on the training split: top distinctive unigrams for No PCL vs. PCL, ranked by class advantage score (Eq. 1).

No PCL (0)			PCL (1)		
Token	**Count**	**Score**	**Token**	**Count**	**Score**
anti	131	3.37	darkness	6	3.46
fund	62	2.63	teresa	6	2.76
released	56	2.53	dole	6	2.76
illegal	354	2.42	hungry	22	2.45
election	86	2.26	christmas	25	2.37
reported	86	2.26	compassion	8	2.32

Table 3: Representative distinctive unigrams by class (from `lexical_analysis.csv`). The score is the magnitude of the smoothed log-rate contrast in (Eq. 1); larger values indicate stronger class-specific lexical signal.

Analysis. Table 3 reports representative examples (counts and association strength). The two classes exhibit different unigram profiles: several tokens show large smoothed log-rate contrast in (Eq. 1), which indicates the presence of class-associated lexical cues in the training split. Tokens associated with **No PCL** (e.g., *election*, *reported*, *illegal*, *released*) resemble informational or institutional/news-style framing, which plausibly corresponds to paragraphs describing events, policy, or factual reporting. In contrast, tokens associated with **PCL** (e.g., *compassion*, *hungry*, *christmas*, and other charity- or sentiment-linked terms) reflect emotive or moralizing language that aligns with common PCL patterns (e.g., pity, benevolence, or saviour narratives). Importantly, some highly distinctive PCL tokens are topic/entity-linked (e.g., *teresa* and *christmas* in Table 3), suggesting that separability may partly reflect recurring story clusters rather than purely pragmatic condescension. This raises a potential shortcut risk: models may learn to associate charity-themed vocabulary with the positive label even when the paragraph is not patronizing.

Impact statement. This lexical analysis directly informs both model development and subsequent evaluation:

- **Shortcut risk and robustness checks:** The presence of highly class-indicative unigrams motivates explicit robustness analyses. In Stage 5, we will report slice-based performance on examples containing strong trigger tokens (e.g., charity/sentiment cues) and inspect whether false positives are disproportionately driven by such vocabulary rather than patronizing intent.
- **Baselines and ablations:** Table 3 indicates strong unigram-level cues, adding a lightweight lexical baseline (e.g., TF–IDF with linear classifier) could quantify how much of positive-class F1 is achievable from surface statistics alone. If this baseline performs strongly, it would indicate shortcut pressure; if it performs poorly, it would justify focusing on deeper contextual modeling.
- **Regularization/augmentation priorities:** Since some high-scoring tokens may function as shortcuts, applying regularization and targeted perturbations could reduce over-reliance on trigger words and improve robustness. Concretely, early stopping and simple lexical perturbations (e.g., token dropout or synonym substitution for highly indicative tokens) may encourage more context-sensitive decision rules, which would ideally reduce false positives driven by charity-themed vocabulary.

3 Baseline Model and Novel Approach Proposal

The project follows SemEval-2022 Task 4 [4], which frames PCL detection as a binary classification problem over paragraphs drawn from the *Don't Patronize Me!* dataset. We adopt the official baseline released by the task organizers, which fine-tunes **RoBERTa-base** [5] for paragraph-level PCL identification and evaluates systems using the F1 score of the positive class (PCL $= 1$), rather than overall accuracy. This baseline provides a clear performance floor for subsequent modelling choices: it achieves $\mathbf{F1} = \mathbf{0.48}$ on the official dev set and $\mathbf{F1} = \mathbf{0.49}$ on the official test set under the positive-class F1 metric.

The official RoBERTa-base baseline provides a useful performance floor, but it remains a relatively minimal fine-tuning recipe: it uses a single backbone, a standard cross-entropy objective, and does not explicitly incorporate (i) the severe class imbalance, (ii) robustness controls against lexical shortcut cues highlighted by the unigram analysis, or (iii) mechanisms to stabilize predictions under threshold sensitivity and run-to-run variance near borderline cases. These limitations motivate an approach that remains compatible with the shared-task evaluation protocol while directly targeting the dataset's empirical failure modes.

3.1 Proposed Approach

We therefore propose an **Impact-Aware Robust Fine-Tuning (IARF)** pipeline that improves over the baseline through three coordinated components:

1. Stronger objectives for imbalanced learning.
2. Lexical-robust data augmentation.
3. Probability-level ensembling with auditability safeguards.

Concretely, we load the official train/dev/test splits with strict order preservation and robust path resolution, and map the dataset's ordinal labels to binary targets using the shared-task convention. We then fine-tune multiple Transformer backbones (e.g., **RoBERTa-base** [5], **RoBERTa-large** [5], **DeBERTa-v3-base** [6]) under controlled seeds, using per-model tuned learning rates and batch sizes with warm-up, weight decay, and a fixed epoch budget. Training uses either standard cross-entropy or a class-weighted focal loss (with $\gamma = 2.0$) with inverse-frequency class weights to emphasize hard positive examples. For robustness, we apply lexical trigger dropout (with default probability 0.2, applied to positive examples) using PCL-indicative tokens derived from lexical analysis. Each run saves aligned artifacts (dev, probabilities, labels, predictions, and a run summary), after which we ensemble selected runs by averaging predicted probabilities, choose the decision threshold on dev via a sweep (0.05 to 0.95, step 0.005), and finally emit `dev.txt` and `test.txt` with one binary prediction per line.

3.2 Rationale and Expected Outcome

The IARF design is intentionally aligned with the dataset properties from EDA and with the shared-task metric:

- Because PCL is rare (with 9.5% of the official training split), cost-sensitive optimization is a natural deviation from standard cross-entropy: class-weighted focal loss up-weights minority positives and concentrates learning on misclassified examples, which is expected to improve positive-class recall without collapsing precision, thereby directly improving the official positive-class F1.
- The unigram analysis revealed strongly class-associated tokens and topic-linked cues, raising the risk that a fine-tuned Transformer may partially rely on lexical shortcuts rather than contextual patronizing intent. The proposed trigger-token dropout operationalizes this insight by perturbing highly indicative tokens during training, encouraging the model to ground decisions in broader context and tone rather than keyword presence, which should improve robustness and reduce shortcut-driven false positives.
- Borderline examples and threshold choice can substantially affect positive-class F1 under imbalance; a fixed 0.5 threshold is therefore not guaranteed to be optimal. IARF addresses this by performing dev-set threshold selection explicitly optimized for positive-class F1, ensuring that the operating point matches the evaluation criterion.
- To reduce run variance and mitigate occasional unstable trainings, we use probability-level ensembling across seeds and backbones, while filtering out degenerate runs via lightweight health checks before ensembling and retaining run manifests for auditability.

Overall, we expect this pipeline to **exceed the RoBERTa-base baseline** on dev and to yield a more stable and reproducible submission: the run matrix enables ablation-style comparisons (objective choice, robustness augmentation, and ensembling), and the ensemble mechanism ensures that if one model family is unstable, a healthy subset can still produce a valid, competitive `dev.txt`/`test.txt` output.

4 Model Implementation

We implement a controlled training matrix over three Transformer backbones (i.e., **RoBERTa-base** [5], **RoBERTa-large** [5], **DeBERTa-v3-base** [6]), using the official train/dev/test splits produced in preprocessing. Ordinal labels are mapped to binary targets following the shared-task convention (i.e., $\mathbf{1}\{\texttt{label} \geq 2\}$). For each run, we fix the random seed and train for 4 epochs with AdamW-style optimization, using warm-up, weight decay, and gradient norm clipping. Models are checkpointed and evaluated at epoch granularity, and we select the best checkpoint according to dev-set positive-class F1 (the shared-task metric). To support reproducibility and auditability, we store per-run artifacts (e.g., dev/test probabilities and run summaries) and apply lightweight health checks before ensembling; the final submission is produced by averaging probabilities across selected runs, tuning the decision threshold on the dev set, and writing `dev.txt`/`test.txt` in the required one-label-per-line format.

Backbone-specific hyperparameters can be found in Table 4, whereas training and inference setting can be found in Table 5. Project repository can be found at: https://github.com/Yi1lan/NLP-Lifecycle-Research. Best model can be found in `outputs/stage4/best_model/best_model_summary.json`, whereas the best ensemble summary can be found in `outputs/stage4/final_ensemble/ensemble_summary.json`.

Backbone	Learning Rate	Train Batch Size	Max Length	Tokenizer
RoBERTa-base	2×10^{-5}	32	probed (192 or 256)	fast
RoBERTa-large	1×10^{-5}	10	probed (192 or 256)	fast
DeBERTa-v3-base	1.8×10^{-5}	24	probed (192 or 256)	slow (default)

Table 4: Backbone-specific hyperparameters used in the model training matrix. The maximum sequence length is selected via a short probe (192 vs. 256) rather than fixed a priori.

Component	Setting
Data + targets	Official train/dev/test ordering preserved; ordinal-to-binary mapping.
Training schedule	4 epochs; epoch-level evaluation/checkpointing; best checkpoint selected by dev positive-class F1.
Optimizer + regularization	AdamW-style optimization; warm-up ratio = 0.1; weight decay = 0.01; max gradient norm = 1.0.
Precision / hardware	CUDA training; bfloat16 enabled when available.
Loss function	Standard cross-entropy (CE) or class-weighted focal loss with $\gamma = 2.0$.
Class weighting	Inverse-frequency binary class weights (optional scale factor; default = 1.0).
Robustness augmentation	Lexical trigger dropout applied to **positive examples only**; default drop probability = 0.2; trigger list derived from Stage 2 lexical analysis (fallback list used if absent).
Random seeds	Seed set used in matrix: {42, 123, 2024, 3407, 777}.
Max-length probe rule	Compare 192 vs. 256 on a representative configuration (RoBERTa + focal + lexdrop); choose 192 if $\mathrm{F1}_{256} - \mathrm{F1}_{192} \leq 0.005$, else choose 256.
Per-run artifacts	Save dev/test probabilities and run manifests (labels/predictions/summary) to enable auditing and ablations.
Ensembling	Average probabilities across selected runs; select threshold on dev by sweeping 0.05 to 0.95 (step 0.005).
Run health checks (before ensemble)	Exclude degenerate runs using: dev F1 ≥ 0.40; dev probability std ≥ 0.01; predicted positive rate in $[0.02, 0.60]$.
Outputs	Emit `dev.txt` and `test.txt` with one binary label per line.

Table 5: Training and inference settings for model implementation, including robustness augmentation, selection criteria, and ensemble decision rules.

5 Evaluation and Error Analysis

Following the shared-task protocol, we generated predictions for the official dev and test splits and exported them as `dev.txt` and `test.txt`, each containing one binary prediction per line in the official input order. We validate formatting and ordering locally: `dev.txt` contains $2,094$ lines and `test.txt` contains $3,832$ lines, all entries are in $\{0, 1\}$, and the order-hash checks pass. Since official test labels are hidden, test-set performance will be determined by the blind evaluation after submission; we therefore report dev-set performance as our local mirror for model selection and analysis.

On the official dev-set mirror, our final submitted ensemble achieves **positive-class F1 = 0.6400** at threshold 0.470, exceeding the provided baseline dev F1 of 0.48. Detail of our final ensemble at be found in repo under `outputs/stage4/final_ensemble/ensemble_summary.json` (10 selected model: **RoBERTa seed 42,**

123, 777, 2024, 3407 and **RoBERTa_large seed 42, 123, 777, 2024, 3407**, probability averaging across non-degenerate runs, with threshold selected on dev) yields the metrics in Table 6.

Metric	Value
Decision threshold	0.470
Precision (PCL=1)	0.6368
Recall (PCL=1)	0.6432
F1 (PCL=1)	**0.6400**
TP/FP/FN/TN	128/73/71/1821
Accuracy	0.9312

Table 6: Final ensemble performance on the official dev-set local mirror (positive class = PCL).

To contextualize this performance, Table 7 summarizes the matrix results grouped by run family. Across families, **RoBERTa** variants are consistently strong (best single-run dev F1 $= 0.6289$), while the **DeBERTa** family collapses and is therefore excluded by health checks; the ensemble improves slightly over the best single run (0.6400 vs. 0.6289) by reducing variance across seeds/models.

Run family	#Runs	Mean F1	Best F1	Mean P	Mean R	Deg.
RoBERTa_large	5	0.5992	**0.6289**	**0.6241**	0.5819	0
b1_RoBERTa	5	**0.6156**	0.6276	0.5832	0.6543	0
b0_RoBERTa	5	0.6124	0.6273	0.5977	0.6352	0
RoBERTa (lex-drop focal)	5	0.6151	0.6219	0.5686	**0.6724**	0
DeBERTa-v3-base	5	0.1741	0.1759	0.0953	1.0000	5

Table 7: Run-family summary across the Stage 4 training matrix (dev-set local mirror). Here, **b0_RoBERTa** is the RoBERTa-base baseline trained with cross-entropy (CE) loss and no lexical dropout, while **b1_RoBERTa** is the same RoBERTa-base setup trained with focal loss and no lexical dropout. Mean P and Mean R denote mean precision and mean recall across runs; **Deg.** is the number of runs flagged as degenerate by Stage 4 health checks.

5.1 Error Analysis and Local Evaluation

At the selected dev threshold (i.e., 0.470), we observe 144 total errors out of examples: 73 false positives and 71 false negatives. This corresponds to an FPR (class 0) of 0.0385 and an FNR (class 1) of 0.3568, indicating that the remaining weakness is predominantly missed positives (minority recall) rather than widespread false alarms. Figure 4 shows the confusion matrix.

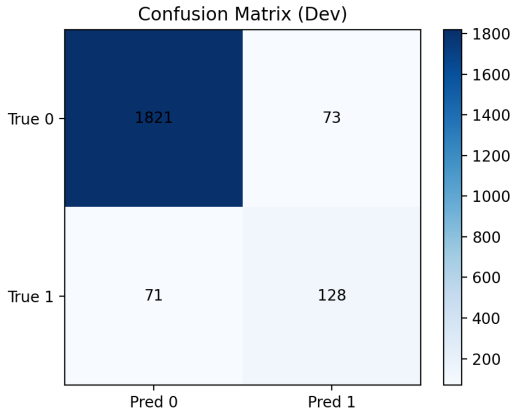

Figure 4: Confusion Matrix for TP/FP/FN/TN Trade-off

We slice dev performance to identify systematic failure regions, obtaining the following observations:

- From Table 8, the model performs strongly on explicit aid-language slices (e.g., *in-need*, *refugee*) but is notably weaker for topics where PCL is often more implicit (e.g., *immigrant*, *women*, *migrant*).

Keyword	Count	#Pos	Recall	F1
Lower-recall keywords				
immigrant	218	7	0.2857	0.4444
women	233	14	0.3571	0.4545
migrant	206	5	0.4000	0.4444
Higher-recall keywords				
in-need	226	33	0.9394	0.7949
refugee	188	13	0.7692	0.7143

Table 8: Slice-based dev evaluation by keyword.

- From Table 9, performance dips for mid-length texts (i.e., 65-128 tokens) relative to shorter examples, suggesting that moderate context length may increase ambiguity without providing sufficiently cues.

Length bin	Count	Recall	F1
$\leq$64	1673	0.6571	0.6502
65–128	386	0.5849	0.6019
$>$128	34	0.8333	0.7143

Table 9: Slice-based dev evaluation by length bin. Mid-length texts (65–128) are the weakest slice.

- From Table 10, trigger-token presence is a double-edged signal: trigger-rich examples are easier (high F1), but the slice also shows higher false-positive tendency, consistent with occasional over-triggering on compassion/help vocabulary.

Slice	Count	Precision	Recall	F1	FPR
No trigger token	2047	0.6067	0.6136	0.6102	0.0374
Has trigger token	46	0.8696	0.8696	0.8696	0.1304

Table 10: Lexical-trigger slice on dev. Trigger-rich examples are easier but also increase false-positive tendency.

- From Table 11, by original severity label, recall increases with severity: the model detects severe PCL (label 4) well, but under-detects borderline cases (label 2), matching the observed FN pattern that subtle condescension is often missed.

Original severity	Count	Recall
2	18	0.2222
3	89	0.5506
4	92	0.8152

Table 11: Recall by original ordinal severity label. Lower severity (label 2) is hardest.

We extracted the highest-confidence dev-set errors form the final ensemble and inspected them qualitatively; representative examples are shown in Table 12. The false positives are dominated by *aid/help narratives* containing explicit trigger phrases (especially the keyword *in-need*, e.g., *help those who are in need*), where the paragraph expresses genuine support but does not clearly encode the superiority/saviour stance that defines PCL. This failure mode is consistent with our earlier lexical analysis warning that charity/sentiment-linked tokens can become shortcut cues and induce over-prediction of PCL. Conversely, the false negatives often occur in *institutional or policy-tones writing* (e.g., government programs, subsidies/taxes) where condescension can be expressed via expert voice or presuppositions rather than overt lexical markers like *help* or *in need*, making these positives harder to detect with primarily lexical-robust fine-tuning. This also reflects the dataset guideline that descriptions of hardship or vulnerability are not automatically PCL; distinguishing *about vulnerable groups* from *patronizing toward vulnerable groups* requires pragmatic cues beyond keyword presence.

ID	Keyword	Err	Gold	Pred	p(PCL)	Excerpt
7414	disabled	FN	1	0	0.057	Cheung said 20 disabled undergraduate students from seven universities will start their eight-week internship in government departments this month, receiving the same salaries as able-bodied colleagues of ...
10326	poor-families	FN	1	0	0.065	A top health official said today that the government could consider subsidies to help poor families pay for healthy food – or imposing taxes on unhealthy products – ...
1870	vulnerable	FN	1	0	0.066	He said: "I think we can consider introducing a negligible telecom surcharge to be entirely borne by the initiator of a call. In order to ..."
10031	in-need	FP	0	1	0.949	Adopt a Mission serves as a platform for the church and for likeminded people to reach out to unemployed families in the communities – whoever is in need ...
8838	in-need	FP	0	1	0.947	"I always consider this job as a gift, being a nurse is a reward and task given by God to help those who are in need ..."
9411	in-need	FP	0	1	0.943	"The boxing community is wonderful at helping those in need, today is a prime example."

Table 12: Representative high-confidence dev-set errors from the manual review pool. False positives are dominated by explicit aid-language (e.g., *in-need*) without clear patronizing stance, while false negatives tend to involve institutional/policy tone where condescension may be implicit rather than lexically marked.

5.2 Decision Calibration and Comparative Diagnostics

Threshold sensitivity. Since the task metric is positive-class F1, we sweep thresholds and select the best dev operating point at 0.470. Figure 5 shows a clear optimum around this value, justifying the use of calibrated threshold selection rather than a fixed 0.5 decision rule.

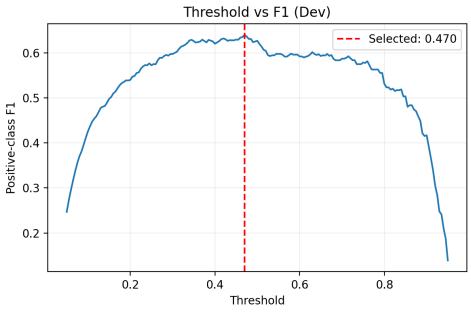

Figure 5: Threshold vs. F1 Curve

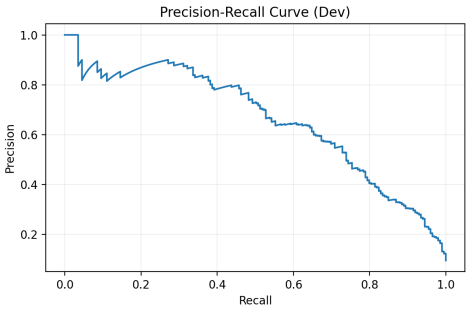

Figure 6: Dev Precision-Recall Curve

Precision-recall trade-off. Figure 6 reports the dev PR curve to visualize how precision degrades as recall increases, highlighting that the operating point reflects an explicit trade-off rather than accuracy-like metric.

Baseline disagreement analysis. We compare baseline vs. final predictions in Table 13. Most examples are stable (both correct 91.59%), while improvements are concentrated in a smaller subset where our model flips labels relative to the baseline, consistent with gains coming from re-balancing positive precision/recall rather than uniformly changing decisions.

Category	**Count**	**Share**
Both correct	1917	91.59%
Both wrong	107	5.11%
Baseline only correct	37	1.77%
Final only correct	32	1.53%

Table 13: Baseline vs final prediction agreement on dev (local evaluation).

Failure mode documentation (DeBERTa collapse). Across our matrix, the **DeBERTa** and, in multiple settings, exhibits degenerate probability outputs (near-constant/saturated predicted probabilities), which translates into very low F1 despite identical data and training pipeline This suggests an optimization/calibration instability specific to DeBERTa under our imbalance-handling recipe. A plausible explanation is that DeBERTa is more sensitive to the interaction between aggressive imbalance mitigation (focal loss with class-weights) and our regularization/perturbation choices (e.g., lexical dropout), which can amplify gradient scaling on minority examples and push logits into saturation early in training, especially in a relatively small, skewed dataset like PCL. We ran a focused diagnosis sweep varying the objective (CE vs. focal), lexical dropout on/off, tokenizer mode, learning rate, and class-weight scale in Table 14; however, these changes did not meaningful recover performance, indicating the issue is not a single toggle but a broader stability mismatch between DeBERTa and our training configuration. Given the repeated collapse and the risk of diluting a stronger system, we exclude DeBERTa from the final ensemble and treat it as a documented failure case; future work could explore DeBERTa-specific stabilizers such as longer warm-up, smaller LR, freezing lower layers, or post-hoc calibration (e.g., temperature scaling) before re-introducing it.

Diagnosis variant	Change vs default
DeBERTa_ce_no_lexdrop	CE loss; no lexical dropout
DeBERTa_focal_no_lexdrop	Focal loss; no lexical dropout
DeBERTa_focal_lexdrop	Focal loss; lexical dropout enabled
DeBERTa_focal_lexdrop_fast_tokenizer	Focal + lexdrop; force fast tokenizer
DeBERTa_focal_lexdrop_lr8e6	Focal + lexdrop; lower LR to 8×10^{-6}
DeBERTa_focal_lexdrop_weight_half	Focal + lexdrop; halve class-weight scale
Mean F1	0.1736474695

Table 14: DeBERTa diagnosis sweep: configurations explored to address the degenerate DeBERTa behaviour observed in the run matrix.

References

[1] Sik Hung Ng. Language-based discrimination: Blatant and subtle forms. *Journal of Language and Social Psychology*, 26(2):106–122, 2007. doi: 10.1177/0261927X07300074. URL https://doi.org/10.1177/0261927X07300074.

[2] Carla Perez Almendros, Luis Espinosa Anke, and Steven Schockaert. Don't patronize me! an annotated dataset with patronizing and condescending language towards vulnerable communities. In Donia Scott, Nuria Bel, and Chengqing Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics*, pages 5891–5902, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.518. URL https://aclanthology.org/2020.coling-main.518/.

[3] Mark Davies. Corpus of news on the web (now) - june 2018. Dataset, University of North Texas Libraries, UNT Digital Library, 2018. URL https://digital.library.unt.edu/ark:/67531/metadc1234358/. Accessed February 15, 2026.

[4] Carla Perez-Almendros, Luis Espinosa-Anke, and Steven Schockaert. SemEval-2022 task 4: Patronizing and condescending language detection. In Guy Emerson, Natalie Schluter, Gabriel Stanovsky, Ritesh Kumar, Alexis Palmer, Nathan Schneider, Siddharth Singh, and Shyam Ratan, editors, *Proceedings of the 16th International Workshop on Semantic Evaluation (SemEval-2022)*, pages 298–307, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.semeval-1.38. URL https://aclanthology.org/2022.semeval-1.38/.

[5] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019. URL https://arxiv.org/abs/1907.11692.

[6] Pengcheng He, Jianfeng Gao, and Weizhu Chen. Debertav3: Improving deberta using electra-style pretraining with gradient-disentangled embedding sharing, 2021.